

Resumeeit — Smart Shortlisting Engine

An AI-powered hiring pipeline: job description to offer, fully explainable at every step

PROJECT REPORT · INTERNLOOM AI HACKATHON · MANIPAL INSTITUTE OF TECHNOLOGY

1. Problem Statement

Given a job description and a batch of 15–18+ resumes, design a system that produces a ranked, explainable shortlist of candidates. The engine must genuinely combine **semantic search** (matching on meaning and context) with **keyword search** (matching explicit skills, tools, and terms) — a system that simply pastes a resume and JD into an LLM and asks for a score does not meet the requirement. For the top 3 candidates, the system must explain which skills matched and which required skills appear to be missing.

2. System Architecture

The product is a two-tier web application: a **React (Vite)** single-page frontend and a **FastAPI (Python)** backend hosting the matching engine. Because a real recruiting run can involve hundreds of resumes, screening does not block on one HTTP request — the frontend submits a job and receives an ID immediately, then polls for progress while the backend parses, embeds, and scores the batch in a background thread. A 204-resume batch was benchmarked end-to-end at under 60 seconds, with the UI showing live phase labels (parsing → embedding → scoring) rather than a frozen spinner.

3. The Matching Engine (Core Requirement)

3.1 Keyword layer

- A curated vocabulary of ~50 technical/soft skills, matched against both the JD and each resume via word-boundary-aware regex, with synonym handling (e.g. "Node", "Node.js", "NodeJS").
- Recruiters can add their own **must-have** or **preferred** keywords beyond the auto-detected list (certifications, internal tool names, phrases like "3+ years Python").
- Every skill — detected or custom — carries an editable **weight multiplier**, so a recruiter can tell the engine that, say, React should count twice as much as Git.

3.2 Semantic layer

- Whole-document similarity is computed with **sentence-transformers (all-MiniLM-L6-v2)** embeddings, with a graceful degradation chain to spaCy word vectors and then TF-IDF + Latent Semantic Analysis if no embedding model is installed — the app never hard-fails on missing ML dependencies.
- The same embedding space is used to **infer missing skills from context**: a resume that mentions "built REST APIs with Node.js and MongoDB" will have **Express** inferred as a likely skill even though the word never appears — directly addressing the hackathon's own worked example.
- Inferred matches are visibly distinguished from explicit matches (in scoring, in explanations, and in on-document highlighting) so nothing is silently treated as equivalent to a literal keyword hit.

3.3 NLP-based candidate extraction

- Regex extraction of email, phone, and LinkedIn handle from resume text.
- **Years of experience:** computed from explicit "N years" mentions or summed employment date ranges — deliberately scoped to the actual experience section (or with the education section excised) so a degree's "2020–2024" is never miscounted as four years of work experience.
- **CGPA:** extracted and normalized to a /10 scale regardless of whether the resume reports a /4.0 or /10 scale.
- **Education level and field of study:** degree parsing (PhD → Master's → Bachelor's → Diploma → High School) with the field of study captured from phrasings like "B.Tech in Computer Science" or "B.Tech Computer Science, XYZ University".
- **Location:** a second, independent ML technique — spaCy named-entity recognition (GPE entities) over the resume header — with a denylist to stop skill/degree words from being mistagged as places, and a deterministic "City, ST" regex fallback for the cases small NER models miss.
- Every extracted field returns *None* rather than a guessed value when it can't be determined, and an unverifiable field never counts as a scoring penalty — only a confirmed shortfall does.

3.4 A fully configurable scoring formula

final score = (semantic weight) × semantic similarity + (1 – semantic weight) × weighted skill coverage, where skill coverage sums each matched skill's weight (required or preferred base weight × that skill's multiplier), crediting inferred-only matches at a configurable fraction of full weight, minus a deduction for any confirmed criteria shortfall. Every constant in this formula — the semantic/keyword balance, base weights, inferred-match credit, and the semantic-match sensitivity threshold — is exposed as a slider in the product, not hard-coded.

4. Explainability

- For the **top 3** candidates, the system generates a plain-language explanation naming exactly which required skills were matched explicitly, which were inferred from context, and which are still missing — assembled from the same computed data used for scoring, not a separate free-text generation step.
- An **offline, rule-based recruiter chatbot** lets a recruiter ask questions like "*Why is Candidate X ranked above Candidate Y?*" in plain English. It parses the question for candidate names or rank references, classifies the intent (comparison, single-candidate explanation, skills, contact, education), and answers entirely from the already-computed ranking data — no network call, no LLM, and it will not validate a false premise (asking "why is the lower-ranked one above" gets corrected, not agreed with).
- Every resume can be opened in a real, in-browser **PDF or DOCX viewer** with highlight overlays drawn directly on the document — green for explicit keyword matches, indigo for inferred semantic matches — so a recruiter can see the actual evidence, not just a summary.

5. Bonus Features Implemented

- **JD bias / narrow-phrasing detection:** a rule-based scan flags gendered or ageist language ("rockstar", "digital native"), experience-level mismatches (e.g. "5+ years" on an internship posting), and requirement lists so long and all-"must-have" that almost no real candidate could satisfy every line.
- **Messy resume handling:** PDF/DOCX parsing failures (scanned images, corrupted files, empty documents) are caught per-file and surfaced individually — one bad file never crashes the batch.
- **Natural-language recruiter chat** and **in-document highlight viewer** (both described above) were explicitly called out as bonus directions in the problem statement.

6. End-to-End Hiring Pipeline (Beyond the Core Requirement)

The core ranking requirement is wrapped in a complete, six-step recruiting workflow, so the shortlist isn't a dead end — it flows all the way to a signed offer:

- **1 • Job Description** — upload the JD as a real PDF/DOCX (rendered as an actual document, not just extracted text), review auto-detected skills, add custom keywords and weights, set education/experience/CGPA criteria, and tune the scoring formula.
- **2 • Resume Upload** — batch upload (PDF/DOCX, tested to 200+ files) and run screening with a live progress bar.
- **3 • Analytics** — score distribution, years-of-experience and CGPA distributions, education breakdown, candidate locations, and the most common strengths and gaps across the whole pool.
- **4 • Shortlist** — a paginated ranked list (10 at a time, not a giant single page) with checkbox selection, a "select top N" shortcut, and the offline recruiter chatbot.
- **5 • Tracker** — a Kanban-style application-management board (Interview / Offer / Hired / Rejected columns) with inline interview scheduling that generates a one-click Google Calendar + Meet invite, and a resume-download action so it can be attached manually where an automated link cannot carry a file.
- **6 • Offer** — generates a real offer letter as a downloadable PDF and a personalized offer email draft, opened in the recruiter's own mail client (nothing is ever sent automatically on the user's behalf).

A full run summary — configuration used, detected skills, the ranked list, and the top-3 explanations — can be exported as a standalone PDF report directly from the Shortlist step.

7. Analytics Dashboard

Beyond the ranked list itself, the Analytics step gives a recruiter a pool-level read before they even open a resume:

- Score distribution across the batch (0–20, 20–40, ... 80–100 buckets)
- Years-of-experience distribution, with unverifiable resumes reported separately rather than as zero
- CGPA distribution (normalized to /10)
- Education-level breakdown (PhD / Master's / Bachelor's / Diploma / High School / unknown)
- Candidate locations, extracted via named-entity recognition
- Most common matched strengths and most common missing-skill gaps across the whole pool
- Any JD bias flags surfaced from the current job description

8. Judging Rubric Alignment

Criterion	Weight	How it's addressed
Effective use of semantic <i>and</i> keyword-based matching	35%	Both signals are computed independently and combined in a transparent, user-tunable formula (§3) — semantic embeddings for meaning/context, regex-based vocabulary matching for explicit terms, with inferred-skill detection bridging the two.

Criterion	Weight	How it's addressed
Quality and sensibility of the ranking	20%	Verified end-to-end on synthetic and real-shaped resume batches, producing a meaningful spread of scores (e.g. 92 → 20 → 0 across a 6-resume test batch) rather than clustered, indistinguishable results.
Accuracy and clarity of top-3 explanations	20%	Explanations are generated from the same matched/inferred/missing-skill data used for scoring, not a separate narrative pass — so what's explained is provably what was scored (§4).
Working end-to-end demo	15%	Full pipeline from JD upload through offer generation is implemented and running; the async job architecture was load-tested at 200+ resumes (§2, §6).
Bonus features	10%	JD bias detection, messy-resume recovery, an offline natural-language recruiter chatbot, and an in-document highlight viewer are all implemented (§5).

9. Tech Stack

Layer	Technologies
Frontend	React 18, Vite, pdf.js (in-browser PDF rendering), mammoth.js (DOCX rendering), jsPDF (offer letters & reports)
Backend	FastAPI, Python, background-thread job queue for large-batch screening
Semantic ML	sentence-transformers (all-MiniLM-L6-v2), spaCy word vectors and NER (en_core_web_sm), scikit-learn (TF-IDF + Truncated SVD fallback)
Document parsing	pdfplumber, python-docx
Data/identity	Firebase Auth & Firestore, localStorage for per-program pipeline state